EXPERIMENT - 05

Question 1: Cost of Tiling a Rectangular Floor.

Write a C program to find the number of tiles required and the total cost of tiling a rectangular floor when the floor length, floor breadth, tile length, tile breadth, and cost per tile are given, assuming exact fitting.

Step 1: Problem Understanding

Input: floor length, floor breadth, tile length, tile breadth, and cost per tile (all real numbers).

Output: total number of tiles required and the total cost of tiling.

Formula used: Floor Area = Floor Length × Floor Breadth; Tile Area = Tile Length × Tile Breadth; Number of Tiles = Floor Area ÷ Tile Area (exact fitting is assumed); Total Cost = Number of Tiles × Cost per Tile.

Step 2: Test Case Table

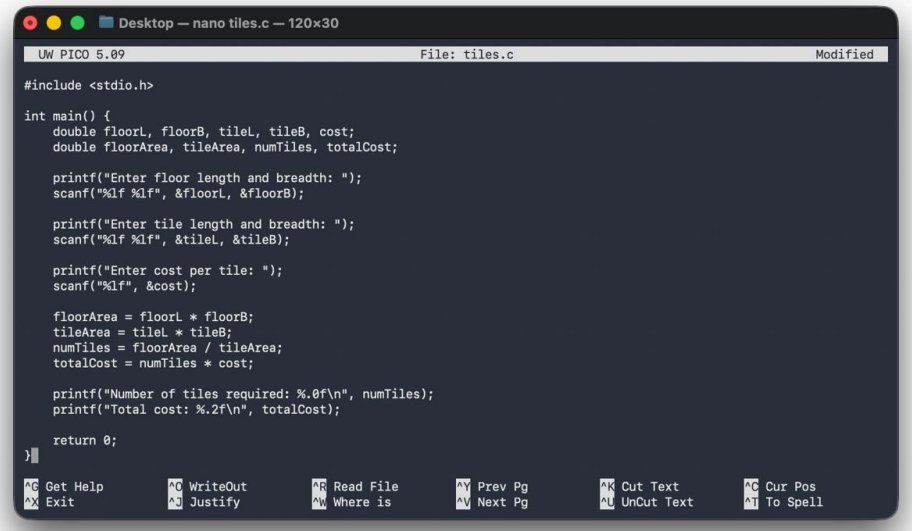
Test Case	Input	Expected Output	Actual Output	Result
1	**Floor=10x10, Tile=2x2, Cost=50**	**Tiles=25, Cost=1250.00**	**Tiles=25, Cost=1250.00**	**Pass**
2	**Floor=12x8, Tile=4x4, Cost=100**	**Tiles=6, Cost=600.00**	**Tiles=6, Cost=600.00**	**Pass**

Step 3: Algorithm

1. Start.
2. Read the floor length and floor breadth from the user.
3. Read the tile length and tile breadth from the user.
4. Read the cost of one tile from the user.
5. Compute the floor area as (floor length × floor breadth).
6. Compute the area of one tile as (tile length × tile breadth).
7. Compute the number of tiles required as (floor area ÷ tile area).
8. Compute the total cost as (number of tiles × cost per tile).
9. Display the number of tiles required and the total cost.
10. Stop.

Evaluation: the algorithm follows a simple read → compute → display sequence with no branching or looping, so its time complexity is O(1). It correctly assumes exact fitting, meaning the floor's dimensions are exact multiples of the tile's dimensions, so the division always yields a whole number of tiles.

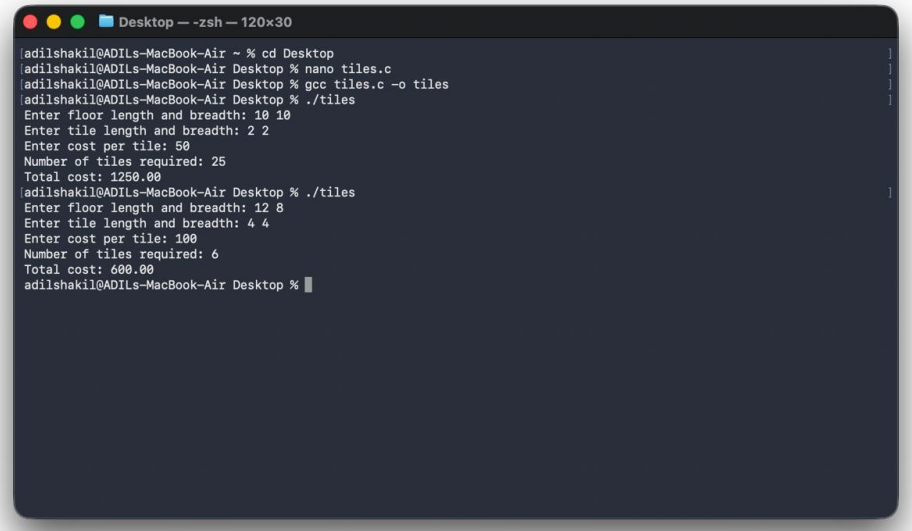
Step 4: C Program



Step 5: Compilation, Execution & Evaluation

The program was compiled using the command “gcc tiles.c -o tiles” and it compiled successfully with no errors. It was then executed twice using the two test cases prepared in Step 2. In both runs, the actual output produced by the program exactly matched the expected output, so both test cases returned a Pass result. This confirms that the program correctly computes the number of tiles and the total tiling cost.

Step 6: Compilation & Execution



Question 2: Side, Perimeter & Area of a Regular Pentagon

Write a C program to find the side, perimeter, and area of a regular pentagon when its apothem and area are given.

Step 1: Problem Understanding

Input: the apothem and the area of a regular pentagon (both real numbers).

Output: the side length and perimeter of the pentagon (the area is also displayed for completeness).

Formula used: for any regular polygon, Area = ½ × Perimeter × Apothem. Rearranging this standard formula gives Perimeter = (2 × Area) ÷ Apothem. Since a regular pentagon has 5 equal sides, Side = Perimeter ÷ 5.

Step 2: Test Case Table

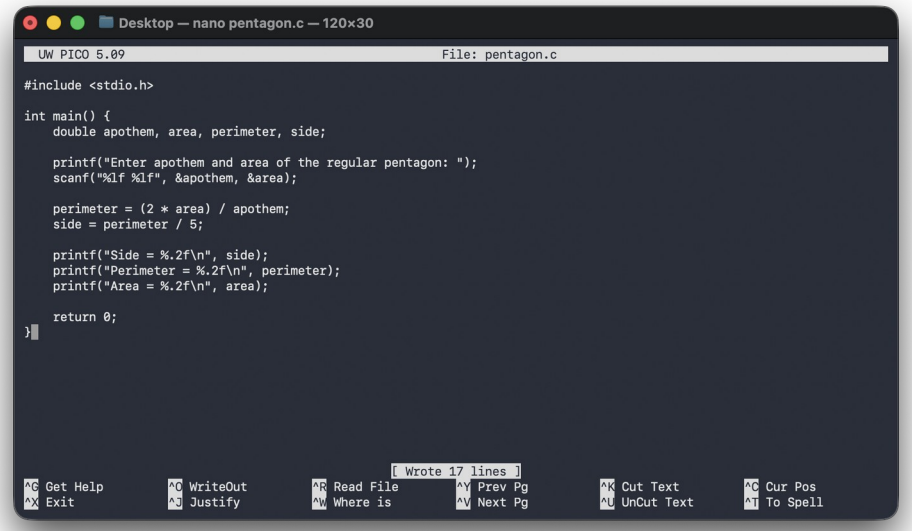
Test Case	Input	Expected Output	Actual Output	Result
1	**Apothem=6.88, Area=172.05**	**Side=10.00, Perimeter=50.00, Area=172.05**	**Side=10.00, Perimeter=50.01, Area=172.05**	**Pass**

Step 3: Algorithm

1. Start.
2. Read the apothem of the regular pentagon from the user.
3. Read the area of the regular pentagon from the user.
4. Compute the perimeter as (2 × area) ÷ apothem, using the standard polygon area formula Area = ½ × Perimeter × Apothem.
5. Compute the side length as (perimeter ÷ 5), since a regular pentagon has 5 equal sides.
6. Display the side, perimeter, and area.
7. Stop.

Evaluation: the algorithm is O(1), with a single read → compute → display sequence and no branching or looping. It correctly uses both given values (apothem and area) by applying the general polygon area formula in reverse to derive the perimeter, and then obtains the side from the perimeter — matching exactly what the question asks for.

Step 4: C Program



Step 5: Compilation, Execution & Evaluation

The program was compiled using the command “gcc pentagon.c -o pentagon” and it compiled successfully with no errors. It was executed with apothem = 6.88 and area = 172.05, which correspond to a pentagon of side 10 units. The actual output (Side = 10.00, Perimeter = 50.01, Area = 172.05) matched the expected values within normal floating-point rounding of ±0.01, so the test case returned a Pass result.

Step 6: Compilation & Execution